City Surveillance Agent — Design Document

Group G041 · MTech AIML · AIMLCZG557 / AECLZG557 · PS9 (Assignment 1) · Source code: `PS9_City_Surveillance_Agent.py` *and* `.ipynb`*; companion files:* `inputPS9.txt`, `outputPS9.txt`, `G041_Contribution.xlsx`*. Within the 4-page limit.*

1. Problem statement and modelling

The Commissioner of Police needs a drone to monitor **every lane** between Chennai tourist landmarks **exactly once**, while individual landmarks may be revisited. We model the city as an undirected weighted graph **G = (V, E)** where V is the set of landmarks and E is the set of lanes. Covering every edge exactly once is the classical **Eulerian path / circuit** (Route Inspection) problem.

By Euler's theorem, such a walk exists iff the graph is connected and has **0 odd-degree vertices** (circuit, start = end) or **exactly 2** odd-degree vertices (path). The map in the problem statement (6 landmarks, 9 lanes) has every degree even → an Eulerian *circuit* exists from any start.

Landmark	Degree	Landmark	Degree
Marina Beach	2	Kovalam Beach	4
Mahabalipuram	4	Muttukadu	2
Vandaloor Zoo	2	Government Museum	4

2. PEAS

Performance	All 9 lanes covered, each exactly once; complete the patrol; minimise revisits/turn-arounds; minimise battery (state expansions and frontier size).
Environment	Static city map (landmarks + lanes), the drone's current landmark, and the set of lanes already used.
Actuators	Drone propulsion / navigation that flies it along the chosen lane to an adjacent landmark.
Sensors	Position sensor (current landmark), map sensor (adjacency + weight), lane-usage tracker (visited lanes).

3. Environment characteristics

Dimension	Value	Justification
Observable	Fully observable	Drone knows the full map and which lanes it has used.
Deterministic	Deterministic	Flying along a chosen lane has a single outcome.
Episodic / Sequential	Sequential	Each move constrains the remaining feasible moves.
Static	Static	The map does not change during the patrol.
Discrete	Discrete	Finite landmarks, lanes and actions.
Single / Multi-agent	Single agent	One surveillance drone.
Known / Unknown	Known	Adjacency and transition rules are given.

4. Heuristic function

State **s = (current, used)** where `used` ⊆ E is the set of lanes already traversed. We use

`h(s) = (|E| − |used|) − ε × uncovered_deg(current)`, `ε = 1 / (|E| + 1)`

The dominant term **|E| − |used|** is the number of lanes still to cover — the standard GBFS "distance-to-goal" estimate, so **lower h ⇒ closer to goal**. The small subtracted term is a tiebreaker: among states with the same `|used|`, those whose **current** landmark still has many unused incident lanes are preferred, because such a landmark offers more options forward and is less likely to strand uncovered lanes.

Properties. h is non-negative, equals 0 only at the goal, and depends only on the current state (no path-cost g(n) is used — that is precisely what makes the algorithm GBFS rather than A*).

5. Data structures (with required overflow / underflow handling)

PriorityQueue — bounded min-heap built on `heapq`. It is the GBFS **frontier**: states are pushed with their h-value and popped in non-decreasing h order. Tied priorities are broken in LIFO order (DFS-like, good for finding *some* Eulerian walk quickly). Both `push` and `pop` report `[PriorityQueue OVERFLOW]` and `[PriorityQueue UNDERFLOW]` and refuse the operation when the capacity is reached / the queue is empty, satisfying the assignment's data-structure error-handling rule.

Graph — adjacency list `dict[landmark → list[(neighbour, weight)]]`. `add_edge` rejects empty names, self-loops and duplicates with explicit error messages.

`frozenset` **of used lanes** — included in each frontier state so the set can be hashed and shared between siblings; gives O(1) "already used?" tests during expansion.

6. GBFS algorithm — how it works in our code

```
push((h(s0), s0)) into frontier    # s0 = (start, empty, [start])
while frontier not empty:
    s = pop_min(frontier)            # lowest h => closest to goal
    if used(s) == E: return path(s)  # goal test
```

```
    for every UNUSED lane (current, nbr):
        s' = (nbr, used ∪ {edge}, path + [nbr])
        push((h(s'), s')) into frontier
return FAILURE
```

Each expansion adds at most `deg(current)` successors, and every successor adds exactly one lane to `used`. Because **|used|** monotonically increases inside the heuristic, any state that can be extended to a complete walk surfaces at the top of the heap before any partial walk that has fallen behind, giving a steady DFS-like dive toward the goal with automatic backtracking through the queue when a branch dead-ends. In our run from *Marina Beach*, GBFS finds a full Eulerian circuit in **10 state expansions** with a peak frontier size of **9** — see `outputPS9.txt`.

7. Complexity

Measured (printed at runtime via `time.perf_counter()` + `tracemalloc`, in `outputPS9.txt`): wall-clock ≈ 1.5×10^{-4} s, peak memory ≈ 9.6 KB, 10 states expanded, peak frontier size 9.

Theoretical (for reference, n = |V|, m = |E|, branching factor b, depth d ≈ m+1): GBFS time = worst case **$O(b^m)$** state expansions; with our state = (current, used), reachable states are bounded by **$n \cdot 2^m$**, so each push/pop is $O(\log(n \cdot 2^m)) = O(m + \log n)$. GBFS space = $O(n \cdot 2^m)$ in the worst case (entire frontier in memory). In practice, the heuristic prunes most of this.

8. Alternate modelling and performance implications

Alternate approach — Hierholzer's algorithm. Build the Eulerian circuit deterministically by walking from the start, removing each lane as it is traversed, and splicing sub-circuits back into the main one whenever the walk closes prematurely. No heuristic is needed.

Aspect	Our GBFS (implemented)	Hierholzer's (alternate)
Time	$O(b^m)$ worst case; tiny in practice with our h	O(n + m) linear
Space	up to $O(n \cdot 2^m)$ for the frontier	O(n + m)
Optimality	Heuristic-guided, returns a valid Eulerian walk	Guaranteed Eulerian walk
Demonstrates	Heuristic state-space AI search (course focus)	Specialised graph algorithm
Generality	Extends to weighted / partial-Euler variants	Solves only classic Eulerian

Conclusion. Hierholzer's is strictly faster asymptotically and would be the production choice; GBFS is required here because the assignment evaluates heuristic search. The trade-off accepted: poorer asymptotic complexity in exchange for a heuristic-driven, easily extensible model.

9. Input / output

`inputPS9.txt` holds the **starting landmark** on a single line. The program prompts `Enter Starting point:` and reads via `input()`; running `python PS9_City_Surveillance_Agent.py < inputPS9.txt` also works for batch testing. The 9-lane Chennai map is hard-coded in `build_chennai_map()` as the assignment permits.

`outputPS9.txt` contains: the starting point, coverage check, all intermediate-path snapshots, the final patrol path (landmarks joined by `->`), and the **measured** time / memory.

Errors handled: empty starting point, unknown landmark, EOF on input, output-file write failure, plus `PriorityQueue` overflow/underflow and `Graph` duplicate / self-loop / empty-name violations.

End of design document — Group G041.